# Verification Is Identification

## Verification Is Identification

A finite structure exists. It is not nothing: a distinction holds. Equality on finite structures is decidable (finitely many components, finitely many comparisons). A decidable test halts, examines finite data, lives in a finite space. If exactly one candidate passes — verification is identification.

One postulate. One loop. 25 proven properties plus information-theoretic bounds. Five theorems — compilation, safety, identifiability, universality, completeness — all derived from the postulate. The inner pipeline is total on valid finite encodings, deterministic under fixed features and protocol, with an append-only comparison cache. Formalized over finite binary trees with decidable equality. Implemented in Python (runtime solver, ARC-AGI tasks) and Swift (compile-time proof: the type checker verifies encoded constraints).

### §0. Derivation

**Postulate.** A finite structure exists.

1. A finite structure exists. *[postulate]*
2. A finite structure is not nothing. *[from 1: if structure = nothing, "a finite structure exists" = "nothing exists" = no postulate. Self-destructs.]*
3. Structure ≠ nothing → a distinction exists between them. *[from 2: "not" IS the distinction.]*
4. Structural equality on finite structures is decidable. *[from 1 + 3: distinction has two sides (step 3). The structure is finite (step 1) → binary distinctions recurse to finite depth → finitely many components. Comparing two finite structures = finitely many component comparisons → terminates → decidable.]*
5. The test halts. *[from 4: decidable ⊂ halting.]*
6. A halting test examines finite data. *[from 1 + 5: finite structure → finite data.]*
7. Finite data of bounded size lives in a finite space F. *[from 1 + 6: the structure has finite size n over finite alphabet Σ (its distinct components). All structures over Σ up to size n form a finite set by combinatorics.]*
8. S = {y ∈ F : COMPARE(y, target) = EQUAL} is computable. *[from 4 + 7: structural equality is decidable (step 4) + finite space (step 7) = enumerate F and check each.]*
9. The structure is representable in F. *[from 1 + 7: it is finite (step 1) and F contains all finite structures up to its size (step 7) → it is in F.]* Under an encoding whose test recognizes it, it passes and enters S.
10. |S| is computable. *[from 8: counting a computed finite set.]*
11. If |S| = 1: the representation that passes IS the structure. *[from 9 + 10: y* ∈ S, |S| = 1, S = {y*}. Any y passing = y*.]*

**Step 11: if |S| = 1, verification is identification.**

**On |S| > 1.** Multiple representations pass. The distinction is insufficient — finer distinctions needed. |S| = 1 is checkable (step 10); if it fails, the system returns |S| > 1.

**On search.** The derivation does not mention how y is found. Search affects whether you find a passing representation, not whether a passing representation is y*.

**On encoding.** The derivation establishes that the structure is representable in F (step 9: it is finite, F contains all finite structures up to its size). Whether the structure enters S depends on the encoding's test — the agent must supply an encoding under which the target passes verification. The theorem holds for any encoding under which |S| = 1.

Verification asks whether a candidate passes. Identification asks which candidate it is. When exactly one candidate passes, passing IS identification: the two predicates are one. Finite space, decidable test, append-only cache make this collapse computable, each derived from the postulate. Once the encoding is fixed, one degree of freedom remains: the order in which facts are checked. Correctness is order-invariant; only efficiency varies (§5.25). An additional degree of freedom — a mutable test, a growing space, a retractable record — breaks steps 4, 7, or 8 respectively, and |S| becomes undetermined.

## §1. The Loop

The system operates as a cycle between two participants:

```
AGENT --→ ENCODING --→ VERIFY --→ PASS: solution identified (Theorem 2)
  ↑                                |
  |-- REJECT(where, why) --|
```

1. **AGENT** provides an encoding: how to represent the task as finite binary trees. The encoding is what makes the system's preconditions hold — finite F, decidable verification. The agent transforms the task into a form the system can certify.
2. **SYSTEM** runs the inner pipeline (§2–§4): encode → extract → compile → execute.
3. **SYSTEM** returns one of:
   - **PASS(f*)**: solution identified. f' = f* by Theorem 2 (§5).
   - **REJECT(basis, position P)**: value at P not in input basis. Agent: change encoding or extend basis.
   - **REJECT(compile, example k)**: rule fails on example k. Agent: add features or change representation.
   - **MISMATCH**: rule compiles but fails on test. Agent: add training examples or change features.
4. **AGENT** reads the diagnostic, modifies the encoding, and re-enters at step 1.

**Inner half** (steps 2–3): total. Always halts. Always returns a definite answer. If PASS: output = f*. If REJECT: no output produced. Proven in §5.

**Outer half** (steps 1, 4): the agent's domain. The loop halts when: - PASS → agent accepts the certified solution. Done. - Agent exhausts grammar → REJECT. No encoding in the grammar produces PASS.

**Totality.** The inner half is a total function on finite inputs: every path terminates at a definite state. Proven in §5.

**Diagnostics.** Each REJECT state carries a structural address — not "failed" but "failed HERE, because THIS." The diagnostic directs the agent: the agent reads where the system's information is insufficient and modifies exactly that.

The following sections detail the inner half: §2 defines the building blocks (what the agent provides), §3 defines the workspace (what the system constructs), §4 specifies the pipeline, §5 proves the guarantees.

## §2. Representation

```
IN:   ∅
OUT:  Σ, COMPARE, Graph
      Invariants: I1 (finite), I2 (closed), I3 (permanent)
```

**2.1. Structures.** A finite structure has parts (distinctions) and ends (boundaries). Each distinction has two sides and a boundary: we notate this as PAIR(A, B). The point where distinction self-terminates is NULL:

- If A, B ∈ Σ then PAIR(A, B) ∈ Σ.
- Finite distinction requires a base: without one, Σ is empty and the postulate is violated. NULL is this base — the point where structure ends.
- Nothing else ∈ Σ. (Finite distinction exhausts all possible finite structures.)

Every element of Σ is a finite binary tree. PAIR is distinction. NULL is its boundary. These are not operations applied to structures — they are what structure is. Σ is the initial algebra of $F(X) = 1 + X^2$: the inductive type of finite binary trees. Any finite structure encodes into it. Here it is not posited — it is what finite distinction produces (steps 1–3).

**2.2. Equality.** Two structures are either structurally identical or not. This is a fact about the structures, not an operation performed on them. We notate this fact as COMPARE : Σ × Σ → {equal, not_equal}:

```
COMPARE(NULL, NULL) = equal
COMPARE(PAIR(A₁,B₁), PAIR(A₂,B₂)) = equal
    iff COMPARE(A₁,A₂) = COMPARE(B₁,B₂) = equal
All other cases → not_equal
```

Structural identity is decidable on finite structures: finitely many components, finitely many comparisons. Projection (accessing parts of a pair) and recursion are traversal of existing structure. All derived operations (ordering, arithmetic, counting) compose from COMPARE and projection via Peano encoding.

**2.3. Graph.** A set G ⊆ Σ × Σ × {equal, not_equal} of recorded comparisons.

**2.4. System invariants.** Each invariant follows from the postulate and representation:

| ID | Invariant | Derivation |
|---|---|---|
| I1 | All structures and sets are finite | From postulate: a finite structure has finitely many components. |
| I2 | All operations reduce to distinction, boundary, identity, access | Distinction (PAIR), boundary (NULL), identity (COMPARE), access (projection). These are structural aspects of finite distinction. No other structural aspect exists. |
| I3 | G is append-only: G' ⊇ G for all future states | From finiteness + I2 (COMPARE is a pure function of its inputs): finite structures are fixed, COMPARE is a pure function of fixed inputs, results cannot change. Retraction is not prohibited — it is impossible. |

I1–I3 are derived from the postulate and representation. Any finite structure is encodable as a finite binary tree; any system that identifies finite structures satisfies I1–I3 under that encoding.

**2.5. Unity.** PAIR is the sole structural primitive: the act of distinction.

- **NULL** is its fixpoint: the pair where left(x) = right(x) = x. Boundary is not a separate symbol — it is where distinction self-terminates. Base case detection: left(x) = x (self-reference), not x = ⊥ (external symbol).
- **COMPARE** is structural recursion over pairs: two trees are identical iff their components are identical at every level, terminating at the fixpoint. Identity is not an external operation — it is what traversal of two structures simultaneously already is.
- **Projection** is destructuring: left(PAIR(A,B)) = A. A direct consequence of distinction having two sides.

One act: distinction. Three derived aspects: where it ends (boundary), whether two histories match (identity), which side you examine (access). We retain NULL and COMPARE as notation — they name structural properties of distinction, not independent entities.

## §3. Problem

```
IN:   Σ, COMPARE, Graph (from §2)
OUT:  Game, F, S, f*
      Encoding adequacy: R4
```

**3.1. Game.** A pair:

- Example pairs: $\{(I_1, O_1), \ldots, (I_t, O_t)\}$ where each $I_k, O_k \subset \Sigma$, finite.
- Test input: $I\_test \subset \Sigma$, finite.

**3.2. Encoding strategy (finite basis selection).** Every output value is selected from a finite basis present in the input. The system does not create new values; it identifies which basis elements belong in which output positions.

Any finite deterministic task can be made basis-selectable by embedding the codomain into the input as a lookup basis. The selection logic lives in the rule, expressed through features.

**3.3. Functions.** Let F = {f : f maps output positions to decoded output values for game G}. Two source-atom selections that produce identical decoded outputs on every input are the same element of F. F is finite (at most |V|^|O| elements, where V = distinct input values) and extensional by construction.

**3.4. Example consistency.** $S = \{f \in F : f(I_k) = O_k \text{ for all } k\}$.

**3.5. Encoding adequacy (R4).** |S| = 1. A decidable property of the encoding: the system computes |S| and reports the result (§3.6). Denote the unique element f*.

|S| > 1: encoding insufficient — the system returns this as a diagnostic (§1). Each additional example can only shrink S, never grow it. |S| = 1: the encoding resolves the task.

**3.6. R4 is decidable.** F is finite (3.3). S is computable by exhaustive check. |S| is therefore computable. (Cost: $O(|F| \cdot T \cdot N^2)$, exponential in |O| but finite.)

**3.7. Applicability.** Consider a task as a relation R(x, y): "y is a valid solution of x". The derivation (§0) starts from one postulate — a finite structure exists. From this, decidable structural equality follows as a theorem (§0 step 4), and three conditions are derived:

1. **Finite basis.** The solution y can be assembled from a finite set of basis elements. *[from §0 steps 1, 6: finite structure → finite data → finite representation]*
2. **Computable verifier.** R(x, y) is decidable: given x and candidate y, verification terminates with PASS/REJECT. *[from §0 step 4: decidable structural equality → decidable check]*

3. **Finite candidate space.** The set F of all basis-assembled candidates is finite. *[from §0 step 7: bounded finite representations form a finite set]*

These follow from the postulate and the derived decidability (§0 steps 1–7). The additional condition — the one that turns verification into identification — is:

4. **Uniqueness (R4).** Exactly one candidate in F passes verification: $|S| = 1$.

**When $|S| > 1$.** Many tasks have multiple valid solutions (e.g., "write any correct sort"). The system returns $|S| > 1$: multiple candidates pass, no unique solution identified. To recover uniqueness, add a selection criterion (shortest, cheapest, lexicographically first). The new relation $R'(x, y) = R(x, y) \wedge y = \text{argmin Cost}(y)$ may yield $|S'| = 1$. Theorem 2 applies to R', not R.

**When $|S| = 0$.** No candidate passes verification. The system rejects: no output produced.

**Real boundary.** The system requires decidable verification and finite F. These are non-negotiable for the inner half. But meeting them is the agent's responsibility (§1): the agent chooses an encoding that MAKES F finite and verification decidable. A task with undecidable verification (e.g., Halting Problem) can be reframed by the agent into a decidable bounded version ("halts within N steps?"). The system's guarantees apply to whatever bounded encoding the agent provides. The boundary is not "is the task inherently finite" but "does the agent's encoding make it finite".

**Naturally basis-selectable tasks** (require no codomain embedding):

| Task type | Why naturally selectable |
|---|---|
| Permutations (sort, shuffle, rotate) | Output = input elements in new positions |
| Spatial transforms (mirror, flip, translate) | Values preserved, positions change |
| Selection / filtering | Output $\subset$ input values |
| Pattern completion | Fill values drawn from existing input |
| Routing / assignment | Items reassigned, not created |
| Constraint satisfaction (sudoku, scheduling) | Solution = input values in valid positions |

**Tasks requiring codomain embedding** (need explicit basis in input):

| Task type | Basis to embed |
|---|---|
| Bounded arithmetic | Result values as lookup table |
| Classification | Label set {A, B, C, …} |
| Bounded program synthesis | Grammar tokens / AST constructors |
| Bounded proof generation | Axioms + inference rules + symbols |
| Structured prediction (parsing, routing) | Finite output vocabulary |
| Translation (bounded vocabulary) | Target language tokens |

These are representative categories. The criterion is structural: does the output consist of values already present in the input (naturally selectable), or must the output vocabulary be explicitly provided (requires embedding)?

## §4. Algorithm

```
IN:   Game, Σ, COMPARE (from §2, §3)
OUT:  f' : candidate function
      result ∈ {PASS, REJECT}
      Invariant: PASS → f' ∈ S
```

**4.1. Encoding.** Represent each game element as a binary tree:

```
encode(0) = NULL
encode(n) = PAIR(encode(n-1), NULL)      for n > 0
atom(r, c, v) = PAIR(PAIR(encode(r), encode(c)), encode(v))
```

The encoding must make all game properties derivable from atoms via COMPARE and projection. No external metadata.

**4.2. Validation.** Verify: ∀ o ∈ O, ∃ i ∈ I such that COMPARE(val(o), val(i)) = equal. If not → not representable in this encoding → REJECT(basis). This is the first diagnostic back to the agent (§1).

**4.3. Equations.** For each example pair, match each output atom to input atoms by value:

```
For each o ∈ O:
    candidates ← {i ∈ I : COMPARE(val(o), val(i)) = equal}
    |candidates| = 1 → unambiguous equation: o ← i
    |candidates| > 1 → ambiguous: hold for resolution
```

Each equation records: this output atom copies its value from this input atom. Ambiguity is not a branch — it is a diagnostic signal. The system knows WHICH atoms are ambiguous and HOW MANY candidates each has.

**4.4. Features.** A feature is a structural predicate over (output position, input position), computable from COMPARE and projection. Examples:

```
same_row(o, i)    = COMPARE(row(o), row(i))
col_mirror(o, i)  = COMPARE(col(o), subtract(max_c, col(i)))
rank(i, I)        = count_less(val(i), vals(I))
```

Features are predefined per domain. The feature set determines what mappings the system can express.

**4.5. Rule extraction.** From unambiguous equations, extract features that are:

- **Consistent**: same value across all unambiguous equations.
- **Informative**: not trivially true for all atom pairs.

The conjunction of consistent, informative features is the preliminary rule. This extraction method is one possible strategy; Theorems 1–2 (§5) hold for any candidate submitted to compilation (§4.7).

**4.6. Ambiguity resolution.** For ambiguous equations, score each candidate by match count against the preliminary rule. Select highest. Ties broken by fixed ordering. Rebuild rule from all equations.

The pipeline is linear: EXTRACT(unambiguous) → RESOLVE(ambiguous, preliminary) → REBUILD(all). Ambiguity resolution is a subroutine inside extraction, not a branch.

**4.7. Compilation.** The verification gate:

```
For each example pair (I, O):
```

```
    If apply(rule, I) ≠ O → REJECT(compile, example k)
```

No candidate passes compilation unless it reproduces all example outputs. REJECT carries the example index — the structural address for the agent.

**4.8. Execution.** If compilation passes, apply rule to I_test:

```
For each output position p:
    select input atom with highest feature-match score
    output[p] ← value of selected atom
```

Output shape must be derivable from input encoding or game definition. Scoring is a heuristic; compilation (4.7) is the guarantee.

## §5. Guarantees

```
IN:   f', result, S, R4 (from §3, §4)
OUT:  Theorems 1-5, Properties, Totality
```

**Theorems**

**Theorem 1 (Compilation).** If result = PASS for candidate f', then f' ∈ S.

*Proof.* PASS means $f'(I_k) = O_k$ for all k (§4.7). This is the membership condition of S. QED

**Theorem 2 (Safety).** If |S| = 1 and result = PASS, then f' = f*.

*Proof.* f' ∈ S (Theorem 1). S = {f*} (R4). f' ∈ {f*} → f' = f*. QED

**Corollary.** Given R4: any compiling output equals f*. Without R4: compilation guarantees example consistency, not uniqueness. Compilation failure: no output.

Safety depends only on compilation (§4.7) and R4, not on how f' was found. The feature search (§4.5–4.6) is one strategy among many — random sampling, exhaustive enumeration, neural network output, or manual construction all produce the same safety guarantee. They differ in how quickly they find a passing candidate, not in what the guarantee means when they do. Safety comes from verification, not search.

**Theorem 3 (Identifiability).** If R4 holds and output values ⊆ input values, then under an exhaustive pair-enumeration schedule, at most $N^2$ COMPARE operations (N = atom count) classify all atom-pair equalities. With all pairs classified, f* is distinguishable from every other candidate via cached verification.

*Proof.* Every function in F maps output positions to input atoms (§3.2). If f ≠ g, they differ on some position p, selecting atoms $i_1 \neq i_2$, so COMPARE($i_1$, $i_2$) = not_equal. I1: N atoms → ≤ $N^2$ pairs. Each COMPARE classifies one pair. I3: classifications are permanent. After $N^2$ COMPAREs, all pairs classified. F is finite (§3.3). Any candidate verifiable against examples using cached results. |S| = 1 → exactly one passes → f* distinguished. QED

**Theorem 4 (Universality).** For any finite deterministic task under a semantics-preserving encoding, there exists an encoding and example set producing an R4-valid game. The framework requires decidable verification over the encoding; the encoding must be chosen so that verification reduces to structural comparisons.

*Proof.* Task finite → encoding yields finite atoms (I1). Basis-selectability: embed codomain into input (§3.2). NULL/PAIR encoding satisfies I2. Graph append-only by I3. Target representable in F (§0 step 9). F is extensional (§3.3): distinct members produce distinct decoded outputs on some input. Providing all possible inputs as examples: any g ≠ f_T differs on ≥1 decoded output → g ∉ S → |S| = 1 → R4. QED.

Practical efficiency depends on encoding choice (§6). The universality is structural: the framework identifies what "proven solution" already requires.

Theorem 4 establishes that R4 is achievable for any finite deterministic task. Theorem 5 (below) establishes that when R4 holds, the system certifies f*. Together: any finite deterministic task admits a certified solution. Practical identification with fewer examples depends on the encoding (§6).

**Corollary (Normal form).** Any finite deterministic task with a computable verifier, a finite basis, and a semantics-preserving encoding whose target is representable in F is a V=I game. Theorem 2 (PASS → f' = f*) applies when R4 is verified. A given implementation's feature set may be too narrow to find a rule for every task in this class; this is an implementation limitation (coverage), not a theoretical one (soundness). Theoretical representability does not guarantee that any fixed feature extractor will find the rule.

**Theorem 5 (Completeness).** If F is finite, S is decidable, and |S| = 1, then certification yields PASS with f' = f*.

*Proof.* F finite → enumerable (§0 step 7). S computable: enumerate F, check each candidate (§0 step 8). |S| = 1 → sole member is f. *Certification: submit f* to compilation (§4.7). f* satisfies all examples (definition of S) → PASS. Theorem 2: PASS + |S| = 1 → f' = f*. QED

Theorems 1–2: soundness (PASS → correct). Theorem 5: completeness (preconditions → PASS). Extraction failure (§4.5–4.6) is a coverage limitation; f* remains certifiable.

**Corollary (Outcome classification).** Every V=I game terminates in one of three states: |S| = 1 (identification, Theorem 2), |S| = 0 (no solution under this encoding), or |S| > 1 (encoding insufficient). *Proof.* |S| ∈ ℕ (§0 step 10). {0, 1, >1} partitions ℕ. QED

**Inner Properties**

**5.1. Termination.** Every algorithm step operates on a finite domain (I1). No step contains unbounded loops. The algorithm terminates for any valid input. QED

**5.2. Complexity.** The algorithm terminates on all finite inputs (5.1). Each step processes a finite domain bounded by atoms, features, and example count. For the reference implementation with T <= 5, N <= 30, F = 5 and bounded-depth Peano encoding, the formal guarantee is termination; empirically, execution completes in negligible time for these bounds.

**5.3. Cache monotonicity.** I3 → G_n ⊆ G_{n+1}. Cached comparisons are never invalidated. `work(game, G) = |needed| - |cached|` estimates uncached comparison work. Monotonically decreasing for overlapping domains.

**5.4. Encoding independence.** Each encoding defines its own function space F and its own f. *Under R4, compilation yields f* for that encoding (Theorem 2). Cross-encoding agreement — that two encodings of the same task produce the same decoded test output — requires an additional premise: both encodings must be semantics-preserving (they represent the same target function on the same domain). This premise is external to the formal system.

**5.5. Totality.** The inner pipeline is linear: ENCODE → VALIDATE → EXTRACT → COMPILE → EXECUTE. Two binary gates (basis validation, compilation) are the only conditional transitions. Every path terminates at one of four states: PASS, REJECT(basis), REJECT(compile), MISMATCH. No execution path diverges, hangs, or reaches an undefined state. *Proof.* Each step operates on finite data (I1, §2). Each comparison is decidable (I2, §2). Ambiguity is resolved by deterministic scoring. QED

### Loop Properties

The following properties hold for the full loop (agent + system). §5.6–§5.7 and §5.9–§5.10 are unconditional (they follow from I3 and I1). §5.8 additionally requires a fixed agent protocol:

**5.6. Monotone information order.** I3 (append-only graph) guarantees that cached comparison results are never retracted. Each loop iteration may add comparisons to G; no iteration removes them. The sequence G_0, G_1, … is monotonically non-decreasing and bounded (finite domain). This induces a total order over system states. *Proof.* G_n ⊆ G_{n+1} by I3. Strict inclusion when new comparisons occur. QED

**5.7. Monotonicity of information.** Each REJECT adds diagnostic information: which encoding failed, at which gate, at which position. The agent's knowledge of "what doesn't work" strictly grows. No loop iteration reduces available information. Combined with I3: both the system's comparison cache and the agent's rejection history are monotonically non-decreasing.

**5.8. Full loop determinism.** If the agent follows a fixed protocol (enumeration order over E), then: same task + same grammar + same protocol → same sequence of iterations → same result. The full loop is a deterministic function from (task, grammar, protocol) to {PASS(f*), REJECT}.

**5.9. Loop termination.** E finite (grammar bounded) + inner half terminates (§5.1) → outer loop terminates. At most |E| iterations, each finite. *Proof.* |E| is finite. Each iteration terminates (§5.1). Finite sum of finite steps is finite. QED

**5.10. Convergence.** The loop tries every encoding in E (finite enumeration). If any encoding's inner run produces PASS, the loop terminates with PASS. If no encoding produces PASS, the loop exhausts E and reports REJECT. No encoding is skipped. An encoding may satisfy R4 yet produce REJECT if the extraction heuristic (§4.5–4.6) fails to find f*. Exhausting E without PASS does not prove that no R4-valid encoding exists in E — only that the extractor failed on all of them. This is an extraction limitation, not a soundness failure. Soundness (Theorem 2) is unconditional: any PASS is correct.

### Compositional Properties

**5.11. Games compose.** Output of game A = input encoding for game B. The composition is a V=I game. *Proof.* Game A output is finite (I1). Game B input is finite. Composition = two total functions in sequence (§5.5). Total ∘ Total = Total. Each game has its own S, its own R4. If both |S_A| = 1 and |S_B| = 1, the composition is verified. QED

**5.12. Games decompose.** If task $T = T_1 \wedge T_2$ (conjunction of independent sub-tasks), then $F\_T \subseteq F_1 \times F_2$. Each sub-game is independently verifiable. *Proof.* If $|S_1| = 1$ and $|S_2| = 1$, then any candidate passing both checks is unique in the product space. Joint PASS = component-wise PASS. QED

**5.13. Sub-game spawn.** At any node in the pipeline, the agent can create a new V=I game to solve a sub-problem (e.g., "which features to try next?"). *Proof.* Sub-problem has finite candidate space (derived from finite parent space). Decidable check (does this candidate work in the parent context?). I1–I3 inherited. The sub-game is a valid V=I instance. Its solution = the encoding for the parent game's next iteration (§1 step 1). QED

### Structural Properties

**5.14. Graph equivalence at every node.** At any point in the computation, the remaining sub-graph from that node to any terminal state has the same V=I structure: finite remaining space, decidable check, append-

only cache. *Proof.* I1 holds at every node (finite data processed → finite remaining). I2 holds (COMPARE unchanged). I3 holds (G at this node ⊇ G at start). All three invariants preserved. QED

**5.15. Structural recurrence across levels.** The system at level N and level N+1 share the same triple structure: (finite space, decidable check, unique solution). The spaces, checks, and solutions differ in content; the structural pattern is identical. *Proof.* Level N: space = F_N (finite), check = compilation (decidable), solution = f*_N. Level N+1: space = E_N (finite), check = run(task, e).status (decidable, §5.5), solution = e*. Both are instances of the V=I triple. QED

**5.16. Finite tower (conditional).** If each meta-level is itself encoded as a finite V=I game, the hierarchy terminates. *Proof.* Level 0: |F| finite. Level 1: |E| finite. Level 2: |grammars over E| = 2^|E|, finite. Each level's space is derived from the previous level's finite space. Finite operations on finite sets produce finite sets. QED

**5.17. Agent-system role duality.** The system at level N serves the same functional role as the agent's tool T2 at level N+1. *Proof.* System at level N: takes candidate, returns PASS/REJECT. This is what an agent at level N+1 calls (T2). The agent's encoding-selection process: takes encoding candidate, returns PASS/REJECT via run(). Same functional interface, different level. QED

**Extended Cache Properties**

**5.18. Cache transferability.** Graph G from solving game A accelerates game B if they share structures. *Proof.* COMPARE(a, b) = structural equality, context-independent. A comparison cached during game A is valid during game B (same structures → same result). I3 → G only grows. work(B, G_after_A) ≤ work(B, G_empty). QED

**5.19. Idempotency.** Running the same encoding on the same task twice produces the same result, with zero new comparisons on the second run. *Proof.* I2 (fixed operations): same input → same output. I3: second run finds all comparisons cached. Same result, zero new work. QED

**Information Properties**

**5.20. Revelation, not creation.** COMPARE(a, b) does not create a fact. The result (equal/not_equal) is structurally determined before the call — the trees are already equal or not. COMPARE reveals what is already the case. *Proof.* COMPARE is a pure function of its inputs (I2). Its output is determined by the structure of a and b, which exist before the call. No randomness, no side effects. The comparison result is a pre-existing structural fact made explicit. QED

**5.21. COMPARE is the sole source of equality-classification facts inside the pipeline.** NULL and PAIR create structure but produce no information about existing structures. Projection (left/right) decomposes structure but produces no equality information. Only COMPARE produces a new equality fact: equal or not_equal. The encoding and feature set carry prior structural information into the pipeline; COMPARE is the sole source of new equality classifications during execution. *Proof.* I2: all operations reduce to NULL, PAIR, COMPARE, projection. NULL and PAIR are deterministic (no input-dependent output). Projection selects a sub-structure (no new information about equality). Only COMPARE takes two inputs and produces a classification. QED

**5.22. S is monotonically non-increasing.** Each COMPARE call can only eliminate candidates from S or leave S unchanged. No COMPARE call adds a candidate. *Proof.* S = {f ∈ F : f matches all examples}. A new COMPARE result adds to G (I3). More facts in G can only reveal that some f fails on some example →

f leaves S. No new fact makes a previously failing f pass (the example it failed on is unchanged). |S_{n+1}| ≤ |S_n|. QED

**5.23. No self-reference.** The system's operations (I2) do not include any operation that takes the pipeline, the graph G, or the system's state as input. The system cannot inspect its own structure, configuration, or execution history from within the inner pipeline. *Proof.* I2 enumerates all operations: NULL, PAIR, COMPARE, projection. None takes the pipeline or G as an argument. The inner pipeline processes encoded data (finite binary trees) and nothing else. QED

**5.24. Encoding-bounded scope.** The inner pipeline receives only the encoded representation. It has no access to the original task, the agent, the grammar, or any information not present in the encoding. Two different encodings of the same task create two separate, non-communicating pipeline executions. *Proof.* The pipeline's input is (encoded game, feature set). All pipeline operations (§4.1–4.8) reference only this input. No step references the agent, the grammar, or any external state. The pipeline is a pure function of its encoded input. QED

**5.25. Order as sole dynamic variable.** For a fixed encoding, task, and feature set, the system's facts are pre-existing (§5.20), operations are fixed (I2), the cache is append-only (I3), the space is finite (I1), and correctness is determined (Theorem 2). The only free parameter is the order in which the agent reveals facts via COMPARE calls. Different orders produce different path lengths through the comparison lattice but reach the same endpoint. Correctness is order-invariant; efficiency is order-determined. I3 is what makes order matter: irreversibility means each ordering commits to a path. Diagnostics (§6) function as order guidance: each REJECT prunes branches the agent need not revisit. *Proof.* §5.20: facts are pre-existing. I2: operations fixed. I3: cache append-only. I1: space finite. Theorem 2: correctness depends on R4 and PASS, not on traversal order. §5.8: same order → same result. §5.10: all complete traversals of E reach the same outcome. The traversal order σ determines iteration count T(σ), which ranges from 1 (valid encoding tried first) to |E| (tried last). QED. *Remark:* The ratio T_min / T_max defines an intrinsic task complexity within the framework. T_min = 1 means the task is trivially distinguishable; T_min = $N^2$ means every atom-pair comparison is needed. This measure is derivable from §5.22 (S-shrinkage rate per COMPARE) and Theorem 3 ($N^2$ upper bound).

*Remark (information-theoretic interpretation).* Identifying f* from F requires $\log_2(|F|)$ bits of information (Shannon). Each COMPARE produces a binary outcome — at most 1 bit of information about f*. Equality when the outcome bisects the remaining candidate space; less when the outcome is predictable from prior comparisons. N atoms yield $N^2$ perspectives (§5.2), which always exceeds $\log_2(|F|)$ — the comparison space is redundant. When candidates differ across d independent dimensions and $d > \log_2(|F|)$, the foundation (minimum claims for identification) is bounded by d, not $\log_2(|F|)$: skipping any dimension leaves indistinguishable candidates. The optimal claim order (§5.25) achieves $\lceil \log_2(|F|) \rceil$ claims when $d \leq \log_2(|F|)$; when $d > \log_2(|F|)$, the dimensional bound dominates.

*Remark (axis alignment).* The monotone properties (§5.6, §5.7, §5.20, §5.22) are simultaneously monotone in the same direction: information grows, rejections accumulate, candidates only leave S, and facts are pre-existing. No execution step can advance one axis while reversing another. This global co-orientation follows from I3 alone: removing append-only permits cache retraction, which re-admits eliminated candidates, breaking §5.22 and misaligning the axes. I3 is thus the alignment generator — the single derived property that forces all monotone axes to co-orient. A consequence: saturation $(1 - |S|/|F|)$ is an incorruptible progress metric. It cannot increase without genuine elimination and cannot decrease at all. At saturation = 1, identification is complete (Theorem 2).

## §6. Interface

The loop (§1) has two participants with distinct functions, guarantees, and boundaries.

### System (inner half)

**Function:** Given an encoding, produce a certified answer or a structural diagnostic.

**Guarantees:** Output equals f* under R4 + PASS (Theorem 2). Compilation failure → no output produced (REJECT). Termination, determinism, and cache correctness are unconditional. Full proofs in §5.

**System boundaries** (non-negotiable — the agent must ensure these hold):

| Boundary | Why | Agent's role |
|---|---|---|
| Verification must be computable | Undecidable R(x,y) → no sound gate | Agent chooses a decidable bounded version |
| Candidate space must be finite | Infinite F → R4 undecidable in general | Agent's encoding MAKES F finite |
| f* is encoding-local | Cross-encoding agreement needs external premise (§5.4) | Agent selects semantics-preserving encodings |
| Correctness is relative to R(x,y) | If verifier doesn't capture semantics, f* may not match intent | Agent designs the verifier |

The system never chooses. It computes. Every output is determined by the encoding. The agent is responsible for making the encoding meet the system's requirements.

### Agent (outer half)

**Function:** Provide encodings. Read diagnostics. Modify and re-submit.

**Agent responsibilities** (the system cannot do these):

| Responsibility | Why the system can't |
|---|---|
| Bound the unbounded | Transform infinite/undecidable tasks into finite/decidable ones by choosing bounds |
| Select encoding grammar E | Requires domain knowledge — what features are relevant |
| Design features | Requires understanding what "same row" or "mirror" means for this domain |
| Decide when to stop | The loop halts when the agent says so |
| Interpret | S |

**Agent boundaries** (what limits the agent's effectiveness):

| Boundary | Consequence |
|---|---|
| Grammar E is finite | Agent can only try encodings in E — if none compile, the task is beyond E |
| Features are predefined per domain | REJECT(compile) may mean "features too narrow", not "task unsolvable" |
| No meta-learning | Agent must design features manually; no automatic feature adaptation |

**Agent Toolset**

Five operations, each derived from the system's structure:

| Tool | Derived from | Function |
|---|---|---|
| T1: ENUMERATE | §5.8 (finite grammar) | List all encodings in E |
| T2: RUN | §1 step 2 (call system) | Execute inner pipeline on one encoding |
| T3: READ | §1 step 4 (parse diagnostic) | Extract gate and structural address from REJECT |
| T4: MAP | §6 REJECT table (above) | Convert address to encoding component |
| T5: MODIFY | Grammar structure (finite components) | Replace component value with alternative |

T1 + T2 = sufficient for correctness (brute enumeration finds f*). T3 + T4 + T5 = sufficient for directed search (diagnostic = gradient). All five = minimal complete agent. T2 IS the system (§5.17): the agent does not contain the system — the agent calls it.

**What REJECT carries**

Each REJECT is a message FROM the system TO the agent with a structural address:

| REJECT type | Structural address | Agent action |
|---|---|---|
| REJECT(basis, value V at position P) | V $\notin$ input basis | Extend basis or change encoding |
| REJECT(compile, example k) | Rule fails on example k | Add features or change representation |
| MISMATCH | Compiles but wrong on test | Add training examples or refine features |
| | S | > 1 (if checked) |

**Encoding Selection**

Given a bounded encoding grammar E = {$e_1$, …, $e_m$}:

```
For each e ∈ E:
    Run Algorithm(encode(task, e))
    If PASS → accept e; stop
If no e passes → REJECT
```

This terminates (E finite, each run terminates by §5.1). Each compiling encoding yields its own encoding-local f* under R4.

**What the grammar captures.** The grammar E defines HOW to represent task elements as binary trees. For grid tasks: which positional attributes to encode, what depth, which Peano mapping. Grammar design is per-domain (grids, sequences, trees), not per-task.

**The separation.** The agent provides the encoding. The system certifies the result. Once the encoding is fixed, everything the system does is proven (§5); the only remaining degree of freedom is the order of verification (§5.25). Everything the agent does is engineering.

## Appendix A: Proof Chain

Every line cites its dependencies. No unsupported claims.

```
DERIVATION (§0):
  A1:  A finite structure exists.
       [postulate]
  A2:  A finite structure is not nothing.
       [from A1: else postulate is vacuous]
  A3:  Structure ≠ nothing → a distinction exists.
       [from A2: "not" IS the distinction]
  A4:  Structural equality on finite structures is decidable.
     [from A1 + A3: finite components → finite comparisons → terminates]
  A5:  The test halts.
       [from A4: non-halting = never distinguishes]
  A6:  A halting test examines finite data.
       [from A5: finite time → finite data]
  A7:  Finite data of bounded size → finite space F.
       [from A6: combinatorics]
  A8:  S = {y ∈ F : COMPARE(y, target) = EQUAL} is computable.
       [from A4 + A7: decidable equality + finite space]
  A9:  The structure is representable in F.
       [from A1 + A7; S-membership requires encoding's test]
  A10: |S| is computable.
       [from A8]
  A11: |S|=1 → verification = identification.
       [from A9 + A10]
  A12: Order of verification is the sole degree of freedom.
       [from A4 + A7 + A8: fixed test, fixed space, permanent facts]
       Mutable test breaks A4; growing space breaks A7;
       retractable record breaks A8.
     [contrapositive: extra DOF → specific step fails → |S| undetermined]

SYSTEM (§2):
```

```
  I1: All structures finite.
      [from postulate: finite components]
  I2: Distinction (PAIR) sole primitive; NULL, COMPARE, proj derived.
     [NULL from finite base case; COMPARE from structural recursion;]
      [proj from destructuring]
  I3: G append-only.
      [from finiteness + purity: results cannot change]

PROBLEM (§3):
  P1: F = functions from output positions to decoded values.
      [§3.2 + extensional]
  P2: |F| ≤ |V|^|O|, finite. F extensional by construction.
      [I1, P1]
  P3: S = {f ∈ F : f matches all examples}.
      [definition]
  P4: R4: |S| = 1. Encoding adequacy measure.
      [§0 step 9]

IDENTIFIABILITY (§5):
  D1: N atoms → ≤ N² pairs.
      [I1]
  D2: Each COMPARE classifies one pair.
      [I2]
  D3: Classifications permanent.
      [I3]
  D4: Under exhaustive pair schedule, N² COMPAREs classify all.
      [D1, D2, D3]
  D5: Any f ∈ F verifiable via cached COMPAREs.
      [D4, P1]
  D6: |S| = 1 → exactly one passes → f* distinguished.
      [P4, D5]

COMPILATION (§4.7):
  C1: Checks f'(I_k) = O_k for all k.
      [§4.7]
  C2: Fail → REJECT.
      [§4.7]
  C3: Pass → f' ∈ S.
      [C1, P3]

ENCODING INDEPENDENCE:
  E1: Each encoding defines its own F, S, f*.
      [encoding-local]
  E2: R4 + PASS → f' = f* for that encoding.
      [S3]
  E3: Cross-encoding decoded agreement requires
      [external premise]
      semantics-preservation (external to proof chain).
```

```
SAFETY (§5):
  S1: Output produced only if PASS.
      [C2]
  S2: PASS → f' ∈ S.
      [C3]
  S3: |S| = 1 → f' = f*.
      [S2, P4]
  S4: R4 + PASS → output equals f*.
      [S1, S2, S3]

UNIVERSALITY (§5):
  U1: Finite task → finite atoms.
      [I1]
  U2: NULL/PAIR encoding → I2 holds.
      [I2]
  U3: Append-only → I3 holds.
      [I3]
  U4: F extensional → distinct g ≠ f_T differs on ≥1 output.
      [P2]
      All inputs as examples → g ∉ S → |S| = 1 → R4.
      [P2, finite domain]
  U5: Requires: f_T representable, computable verifier,
      [external premises]
      finite basis.

COMPLETENESS (§5):
  CM1: F enumerable.
       [A7, I1]
  CM2: S computable by enumeration + membership check.
       [A8, P3]
  CM3: |S| = 1 → f* found.
       [CM2, P4]
  CM4: f* satisfies all examples → compilation succeeds.
       [CM3, C1]
  CM5: PASS + |S| = 1 → f' = f*.
       [CM4, S3]
  CM6: Outcome ∈ {|S|=0, |S|=1, |S|>1}. Exhaustive.
       [A10: |S| ∈ ℕ]

INCOMPLETENESS:
  L1: Feature search may fail to find f' ∈ S.
  L2: Compilation rejects.
      [C2]
  L3: Rejection produces no output.
      [S1]
  L4: Exhaustive enumeration of P2 would be complete.
      [P2, D5]
```

```
TERMINATION:
  T1: Each step domain finite.
      [I1]
  T2: No unbounded loops.
      [§4]
  T3: Terminates.
      [T1, T2]

LOOP (§1):
  G1: Grammar E finite.
      [bounded grammar premise]
  G2: Each encoding run terminates.
      [T3]
  G3: Search terminates.
      [G1, G2]
  G4: Compiling encoding yields encoding-local f* under R4.
      [S3]
  G5: Per-task enumeration is finite; grammar adequacy is external.
      [G1, G3]

TOTALITY (§1):
  TOT1: Inner pipeline is linear:
        ENCODE → VALIDATE → EXTRACT → COMPILE → EXECUTE.
        [§4]
 TOT2: Two binary gates (basis, compilation) are the only conditionals.
        [§4.2, §4.7]
  TOT3: Every path terminates at:
        {PASS, REJECT(basis), REJECT(compile), MISMATCH}.
        [T3, TOT2]
 TOT4: No execution path diverges, hangs, or reaches undefined state.
        [TOT1, TOT2, TOT3]

LOOP PROPERTIES (§5.6–5.10):
  LP1: G_n ⊆ G_{n+1} (arrow of time).
       [I3]
  LP2: Diagnostic information monotonically non-decreasing.
       [LP1, C2]
  LP3: Full loop deterministic (given fixed protocol).
       [TOT4, G1]
  LP4: Loop terminates in at most |E| iterations.
       [G1, T3]
  LP5: If valid encoding exists in E, loop finds it.
       [LP4, finite enumeration]

COMPOSITIONAL (§5.11–5.13):
  CO1: Output of game A = valid input for game B.
       [I1]
```

```
  CO2: Total ∘ Total = Total.
       [TOT4, CO1]
  CO3: Games decompose: T = T1 ∧ T2 → verify independently.
       [CO2, P4]
  CO4: Sub-game spawn: any node can create V=I sub-game.
       [I1, I2, I3 inherited]
  CO5: Sub-game solution = parent's next encoding.
       [CO4, CO1]

STRUCTURAL (§5.14–5.17):
  ST1: I1, I2, I3 hold at every node in computation graph.
       [invariant preservation]
  ST2: Sub-graph from any node = V=I instance.
       [ST1]
  ST3: Level N and N+1 share same triple structure.
       [ST2, G1; structural recurrence]
  ST4: Tower terminates when each level explicitly encoded.
       [I1: finite at each level; conditional]
  ST5: Agent at level N = candidate at level N+1.
       [ST3; same role, not identity]
  ST6: System at level N = agent tool T2 at level N+1.
       [ST5; functional duality]

CACHE (§5.18–5.19):
  CA1: COMPARE is context-independent.
       [I2]
  CA2: G from game A valid for game B.
       [CA1, I3]
  CA3: work(B, G_after_A) ≤ work(B, G_empty).
       [CA2]
  CA4: Same encoding twice → same result, zero new work.
       [LP3, I3]

INFORMATION (§5.20–5.25):
  IN1: COMPARE reveals pre-existing facts (revelation).
       [I2: pure function]
  IN2: COMPARE is sole source of equality facts.
       [I2: only COMPARE classifies]
  IN3: S monotonically non-increasing.
       [I3: new facts only eliminate]
  IN4: No self-reference in pipeline.
       [I2: ops don't take pipeline as arg]
  IN5: Pipeline sees only encoded representation.
       [pure function of input]
  IN6: ORDER is sole dynamic variable.
       [IN1, I1, I2, I3, Thm 2]

INFORMATION-THEORETIC (remark):
```

```
  IT1: Identification requires log₂(|F|) bits.
       [I1, Shannon]
  IT2: Optimal order: ⌈log₂(|F|)⌉ claims.
       [IN6, IT1]
  IT3: N atoms → N² perspectives.
       [D1, D2]
  IT4: N² ≥ log₂(|F|) (comparison redundancy).
       [IT3, IT1]
  IT5: f* at level N = substrate at level N+1.
       [CO1, ST3, ST6]
  IT6: Foundation ≥ separating dimensions d.
       [I1, A4: skip d → |S|>1]

AXIS ALIGNMENT (remark):
  AX1: All monotone properties co-oriented.
       [I3 → §5.6, §5.7, §5.22; I2 → §5.20]
  AX2: Saturation = incorruptible progress metric.
       [AX1, §5.22]
  AX3: I3 is the alignment generator.
       [without I3 → §5.22 breaks → axes misalign]
```

## Appendix B: Reference Implementation

The implementation is modularized to reflect the formal separation of concerns:

1. **`primitives.py` (§2):** Peano encoding and structural equality. Domain-agnostic foundation.
2. **`engine.py` (§4 core):** Rule extraction, ambiguity resolution, match scoring. Operates solely on structural predicates — never touches cells, grids, or Peano.
3. **`representation.py` (§3–§4 domain):** Grid cells, grids, features, equations, execution, and compilation.
4. **`tasks.py` (Data):** Task definitions in ARC-AGI format.
5. **`main.py` (Pipeline):** Connects representation, engine, and data.

```
Encoding:    Peano (0 → NULL, n → PAIR(n-1, NULL))
Atoms:       PAIR(PAIR(row, col), value)
COMPARE:     Python == on nested tuples
Features:    5 (same_row, same_col, row_mirror, col_mirror, rank)
Compilation: full example verification before test execution
```

Verified tasks: fill minority, fill majority, horizontal flip, vertical flip, identity. 5/5 compiled and solved. These are demonstrations; they do not prove coverage of the full task class. The task format follows ARC-AGI (Abstraction and Reasoning Corpus): input-output grid pairs where the transformation rule must be inferred from examples.

## Appendix C: Type-Level Proof

The Python implementation (Appendix B) verifies at runtime. A stronger form makes the compiler itself the verifier for encoded type constraints: if the program compiles, the encoded rule constraints are proven. No runtime check needed for those constraints.

In Swift, Peano naturals are types (`Z`, `S<N>`). Grid features become type constraints:

```swift
// SameRow requires: In.Row == Out.Row  (compile-time check)
enum SameRow<In: CellType, Out: CellType>: Rule
    where In.Row == Out.Row { ... }

// ShiftRight requires: S<In.Col> == Out.Col
enum ShiftRight<In: CellType, Out: CellType>: Rule
    where In.Row == Out.Row, S<In.Col> == Out.Col { ... }
```

The compilation gate (§4.7) becomes literal:

```swift
// CompilationGate<R> compiles ⟺ R's constraints are satisfiable.
// If it compiles, the rule is verified. If not, the compiler rejects.
enum CompilationGate<R: Rule>: Gate {
    // Compilation of this enum IS the verification.
    static func verify() {}

    // Type inference determines the output. No search needed.
    static func identify(input: R.Input.Type) -> R.Output.Type {
        return R.Output.self
    }
}
```

Demonstration:

```swift
typealias In  = Cell<N1, N1, N5>  // row=1, col=1, val=5
typealias Out = Cell<N1, N2, N5>  // row=1, col=2, val=5

// This line compiles ⟺ ShiftRight ∧ SameValue holds for In → Out.
typealias Mapping = And<ShiftRight<In, Out>, SameValue<In, Out>>
let gate = CompilationGate<Mapping>.self  // Compilation = PASS
```

Change `Out` to `Cell<N1, N3, N5>` and the program does not compile. The compiler rejects the rule. `PASS = compilation success. REJECT = compilation error.`

Verification and identification collapse into one operation for the encoded constraints: the compiler checks constraints (verification) and determines the unique satisfying type assignment (identification). This is §0 step 11 executed by the compiler for the specific constraints encoded as types. Full identification of the task's target function additionally requires R4 (§3.5) and a semantics-preserving encoding (§5.4).

Source: Python (runtime solver, ARC-AGI tasks) and Swift (compile-time proof). Successful compilation confirms all encoded constraints are verified.